

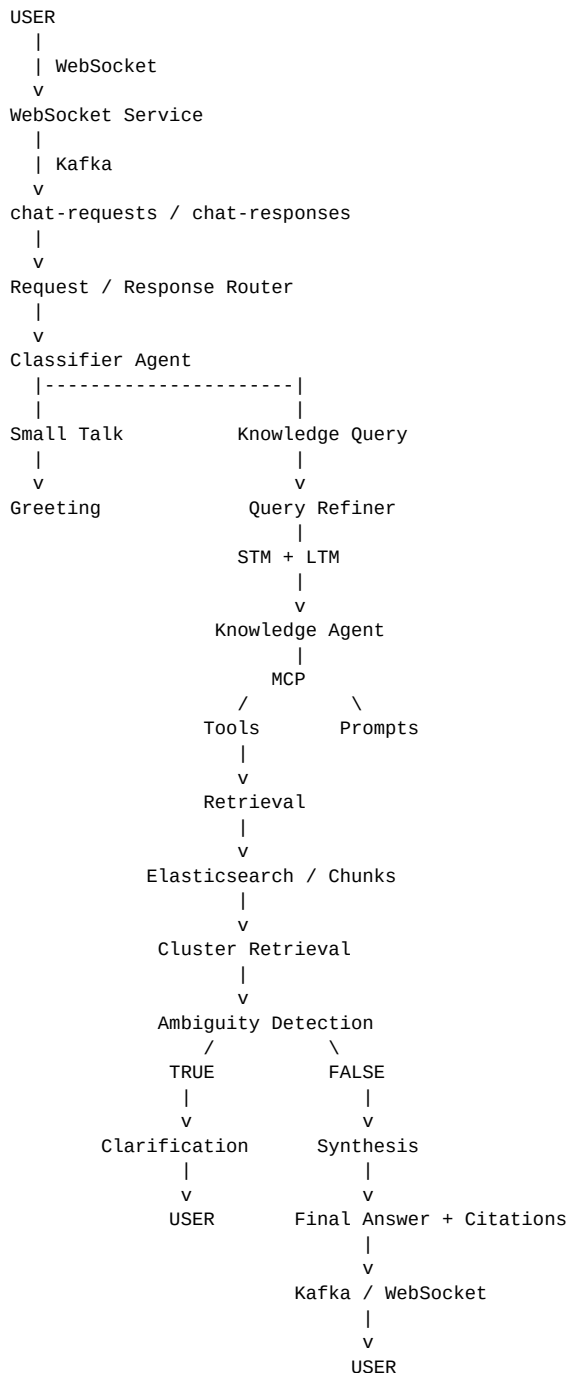
Previous Retrieval + WebSocket Architecture

Detailed reference guide covering the previous project's WebSocket, Kafka, routing, classification, memory, MCP, retrieval, ambiguity handling, clarification, and synthesis flow.

Core takeaway

The WebSocket service is not the retrieval engine. It is the communication and request/response infrastructure around the retrieval and agent system. The complete architecture combines persistent WebSocket communication, Kafka-based asynchronous messaging, request routing, classification, conversational memory, query refinement, MCP-based tools, retrieval, ambiguity detection, clarification loops, and answer synthesis.

1. The Whole Architecture



The system is easier to understand as five layers:

- **Communication:** WebSocket, ConnectionManager, Kafka.

- **Orchestration:** Request/Response Router, Classifier, Query Refiner, Knowledge Agent, Final Answer Collector.
- **Retrieval:** semantic/keyword/hybrid search, cluster retrieval, ranking and filtering.
- **Knowledge/storage:** Elasticsearch, document chunks, metadata, embeddings, ACL information.
- **Memory:** Short-Term Memory (STM) and Long-Term Memory (LTM).

2. Why WebSocket?

A normal REST interaction is request → response → request → response. A conversational application benefits from a persistent, bidirectional channel. The client establishes one WebSocket connection and exchanges multiple messages over it.

```
HTTP / REST:
Request 1 -> Response 1
Request 2 -> Response 2
Request 3 -> Response 3

WebSocket:
Persistent connection
|
+-- User message
+-- Backend response
+-- User message
+-- Clarification
+-- Final answer
```

This is useful when the backend may take several seconds or longer because retrieval, LLM calls, MCP tools, ambiguity checks, or multiple agents may be involved.

3. ConnectionManager — Why It Exists

```
class ConnectionManager:
    def __init__(self):
        self.active_connections = {}
```

The manager maintains a mapping between a conversation/session identifier and its live WebSocket connection.

```
{
    "abc123": websocket_object,
    "xyz789": websocket_object,
    "user456": websocket_object
}
```

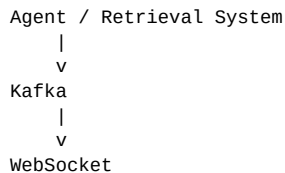
When an asynchronous backend response arrives, the service inspects session_id and routes the response to the correct live WebSocket connection.

```
Kafka response
|
| session_id = abc123
v
ConnectionManager
|
v
active_connections["abc123"]
|
v
WebSocket
|
v
User
```

4. Kafka — Why It Sits Between WebSocket and Agents

The system could call retrieval and agents directly from the WebSocket handler, but that would tightly couple the client-facing layer to potentially slow backend processing. Kafka provides an asynchronous message backbone.

```
WebSocket
|
v
Kafka
|
v
```



This separation can provide buffering, asynchronous processing, consumer groups, partitioning, independent scaling, fault isolation, and message replay depending on the Kafka design.

5. Kafka Producer — Request Flow

The producer sends a user request to the chat-requests topic. The request normally carries the session identifier and user message.

```

producer.produce(
    "chat-requests",
    key=session_id,
    value=json.dumps({
        "session_id": session_id,
        "message": data["message"]
    })
)

```

Using session_id as the Kafka key can help associate messages from the same session with the same partition, supporting ordered processing for that key within a partition.

6. Kafka Consumer — Response Flow

```

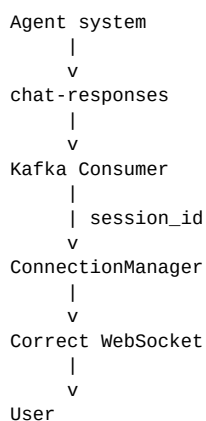
consumer.subscribe(["chat-responses"])

data = json.loads(msg.value().decode())

await manager.send(
    data["session_id"],
    data
)

```

The consumer listens for completed responses. Once a response arrives, session_id is used to find the correct WebSocket connection. The WebSocket layer therefore does not need to understand how retrieval or the agent generated the answer.



7. Request / Response Router

The router coordinates incoming requests with asynchronous processing and responses. Conceptually it maintains a pending request/future for a session or request identifier.

```

session_id = "ABC"

pending_requests["ABC"] = future

... agent processing ...

future -> completed
|

```

response for ABC

A Future represents work that has not completed yet. This is useful when the agent pipeline is asynchronous and may take an unpredictable amount of time.

8. Why Not Call the Agent Directly?

Direct coupling:

```
WebSocket
|
v
Retriever
|
v
LLM
|
v
WebSocket
```

Decoupled:

```
WebSocket
|
v
Kafka
|
v
Agent Runtime
|
+-- Retrieval
+-- LLM
+-- MCP
+-- External tools
|
v
Kafka
|
v
WebSocket
```

The decoupled design allows the client-facing service and agent workers to scale, fail, and evolve more independently.

9. Classifier Agent

The classifier determines what kind of request it has received. The key architectural distinction is between small talk and a knowledge/retrieval query.

```
User: "Hi"
|
v
Classifier
|
v
SMALL_TALK
|
v
Greeting

User: "What is the AML investigation process?"
|
v
Classifier
|
v
KNOWLEDGE_QUERY
|
v
Query Refiner / Retrieval
```

This avoids unnecessary retrieval work. A greeting does not need embeddings, Elasticsearch search, document retrieval, or answer synthesis from enterprise documents.

10. Query Refinement and Conversational Context

A follow-up question may be incomplete when considered in isolation. Query refinement uses previous conversation context to make the query retrieval-ready.

```
Previous:
"What is AML investigation?"

User:
"What about escalation?"

Raw query:
"What about escalation?"

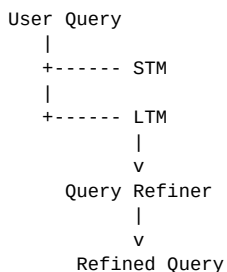
Possible retrieval-ready query:
"What is the escalation process in an AML investigation?"
```

The architecture shows STM and LTM feeding into this stage.

11. STM vs LTM

Short-Term Memory (STM) represents recent conversational context. It helps interpret immediate follow-up questions and references.

Long-Term Memory (LTM) represents information intended to persist beyond the immediate conversational window, depending on the application's memory policy.



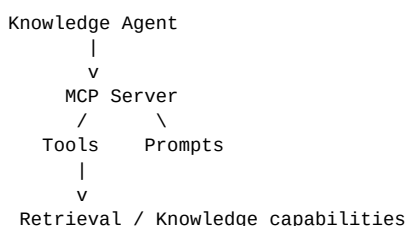
12. Knowledge Agent

The Knowledge Agent is an agentic layer around knowledge access. It is not necessarily the low-level retriever itself. It can decide what knowledge/tool operation is needed, call tools, and coordinate the path to an answer.

This differs from a simple pipeline where every query always executes exactly one predefined search.

13. MCP — Why It Appears in the Architecture

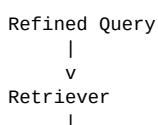
The architecture shows the Knowledge Agent communicating with an MCP server exposing tools and prompts. MCP provides a standardized interface for exposing capabilities to an agent.



Possible tools could include document search, metadata search, cluster retrieval, document lookup, or other enterprise knowledge operations. The exact tool set depends on the implementation.

14. Retrieval — The Core Knowledge Step

For a knowledge query, the refined query is passed into retrieval. Retrieval's job is to find evidence from the indexed knowledge base.



```

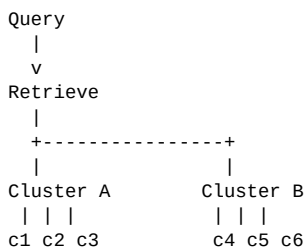
+-- Embedding / semantic search
+-- Keyword / lexical search
+-- Hybrid search
+-- Filtering
+-- Ranking / reranking
|
v
Relevant chunks

```

This is the part most directly related to your current retrieval work: the document ingestion/chunking/embedding/indexing pipeline supplies the searchable knowledge base, and retrieval turns a user query into relevant chunks.

15. Why Cluster Retrieval?

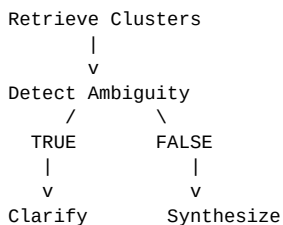
The previous architecture did more than simply return top-k chunks. It groups or reasons about retrieved information as clusters. A cluster can represent a coherent set of chunks related to one topic, document, process, or concept.



This allows the system to reason not only about individual chunk scores but also whether the retrieved evidence points clearly to one interpretation or several competing interpretations.

16. Ambiguity Detection

Ambiguity detection is an important safeguard. The system checks whether retrieved evidence is sufficiently clear before generating a confident answer.

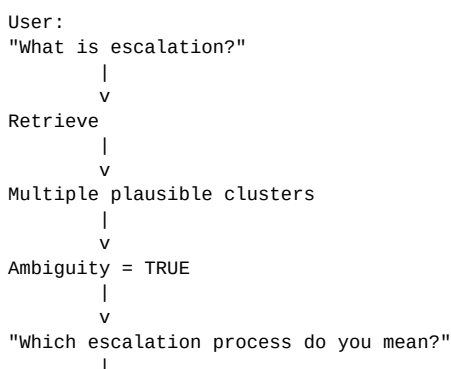


The architecture diagram indicates conditions such as no useful clusters, weak retrieved clusters, or competing clusters with similar scores. It also shows a score threshold around 0.70. The exact threshold and scoring behavior are implementation-specific and should be verified in the retrieval code.

Example: if the query is "What is the escalation procedure?" and retrieval finds similarly relevant AML, fraud, and compliance escalation clusters, the system may not have enough information to choose one interpretation.

17. Clarification Loop

When ambiguity is detected, the system generates a clarification question instead of forcing an answer.



```

      v
User:
"AML escalation"
  |
  v
Refined retrieval cycle

```

This makes the system conversational and reduces the chance of confidently answering the wrong interpretation.

18. Synthesis When Retrieval Is Clear

When ambiguity is false, retrieved chunks are passed to a synthesis step. The synthesis model uses the evidence to produce the final answer, ideally with citations or source references.

```

Query
|
v
Retrieved chunks
|
v
Ambiguity = FALSE
|
v
Synthesis Tool
|
v
Final answer + citations

```

19. FinalAnswerCollector

The FinalAnswerCollector acts as the point where the result of the agent/retrieval workflow is assembled for delivery back to the client-facing system.

```

{
  "session_id": "abc123",
  "answer": "AML investigations are ...",
  "citations": [
    {
      "document": "AML_Policy.pdf",
      "page": 12
    }
  ]
}

```

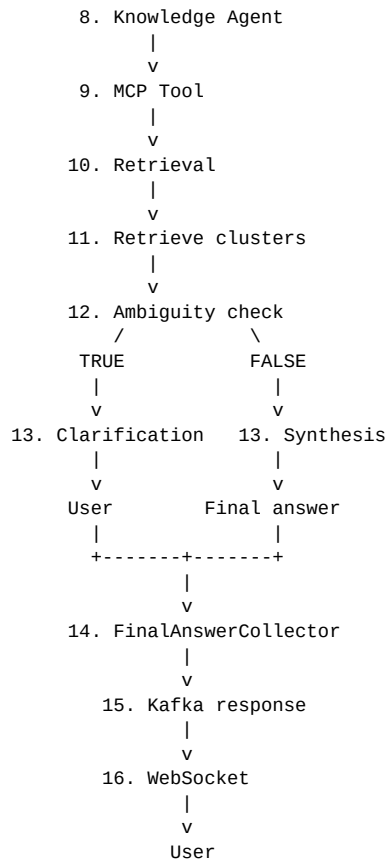
That result can then be published to chat-responses and delivered to the user's WebSocket session.

20. Full Request Lifecycle

```

1. User sends a message
  |
  v
2. WebSocket receives JSON
  |
  v
3. Session ID is attached / preserved
  |
  v
4. Request is published to Kafka
  |
  v
5. Request/Response Router receives it
  |
  v
6. Classifier determines request type
  |
  +---- Small talk ----> Greeting ----> Final Answer
  |
  +---- Knowledge query
        |
        v
        7. Query Refiner
          |
          +-- STM
          +-- LTM
          |
          v

```



21. Why the Architecture Looks Complicated

Do not equate the entire diagram with retrieval. It combines several concerns that can be independently implemented.

- **Retrieval:** Query → search → relevant chunks.
- **RAG:** Retrieval → context → LLM → answer.
- **Conversational RAG:** conversation context → query rewriting → retrieval → answer.
- **Agentic RAG:** agent → tool selection → retrieval → reasoning → possible additional tools → answer.
- **Previous enterprise design:** agentic RAG + memory + ambiguity handling + MCP + Kafka + WebSocket + request correlation.

22. How It Maps to Your Current Project

CURRENT DOCUMENT / RAG SIDE

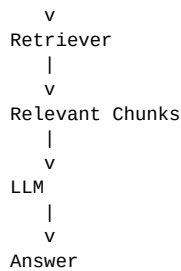
```

Documents
|
v
Extraction
|
v
Chunking
|
v
Embedding
|
v
Elasticsearch
  
```

CURRENT RETRIEVAL SIDE

```

User Query
|
v
Query Processing / Refinement
|
  
```

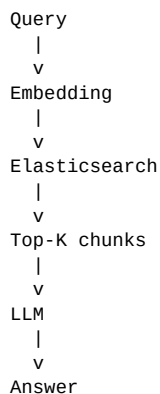


The previous architecture wraps a larger orchestration and communication system around this core retrieval path.

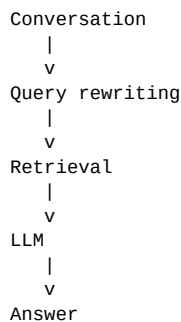
23. Recommended Mental Model for Your Current Retrieval

Do not try to reproduce the entire previous enterprise architecture at once. Build it in layers.

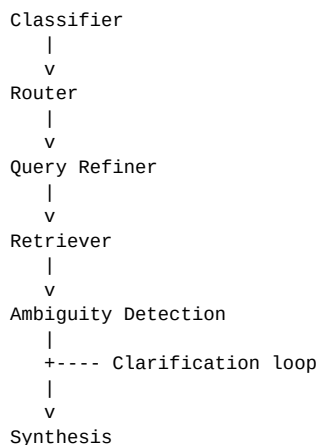
Level 1 — Basic RAG



Level 2 — Conversational RAG



Level 3 — Agentic RAG



Only after the core retrieval flow is stable should enterprise infrastructure such as Kafka, WebSocket, MCP, and multiple agents be added where the application genuinely requires them.

24. Important Implementation Notes From the Previous Code

Kafka bootstrap address: the previous code used a localhost-style Kafka address. That can be correct when the process runs directly on the host. Inside Docker, localhost refers to the current container, so an internal Docker network normally uses the Kafka service name, such as `kafka:9092`, if that is how the compose network is configured.

Blocking Kafka poll: a synchronous `Consumer.poll()` inside an async function can block the async event loop. For production, isolate blocking Kafka work appropriately.

Producer delivery: `produce()` is asynchronous. Calling `flush()` for every message can hurt throughput; production systems normally use delivery callbacks/error handling and controlled flushing.

25. Interview / Design Explanation

WebSocket: maintains a persistent bidirectional connection for conversational interaction.

Kafka: decouples the client-facing communication layer from asynchronous agent processing and provides scalable message transport.

Session ID: correlates asynchronous responses with the correct user's WebSocket session.

Router: orchestrates requests and manages asynchronous request/response correlation.

Classifier: determines whether retrieval is necessary, avoiding unnecessary RAG calls for small-talk queries.

Query Refiner: uses conversational context from STM/LTM to transform follow-up questions into retrieval-ready queries.

Knowledge Agent: decides how to use available knowledge/retrieval capabilities.

MCP: provides a standardized tool interface between the agent and backend capabilities.

Retriever: finds relevant document chunks from the indexed knowledge base.

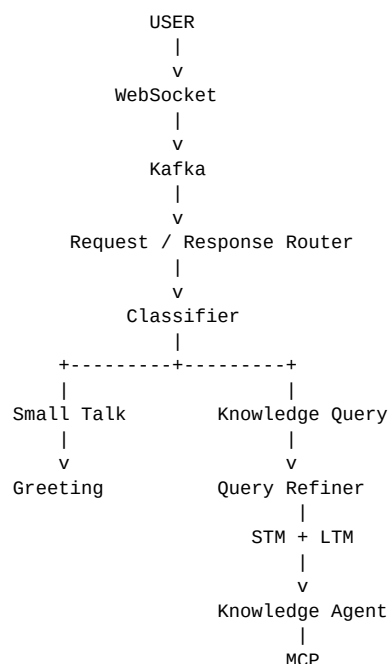
Ambiguity detector: prevents the system from confidently answering when retrieved evidence is insufficient or competing.

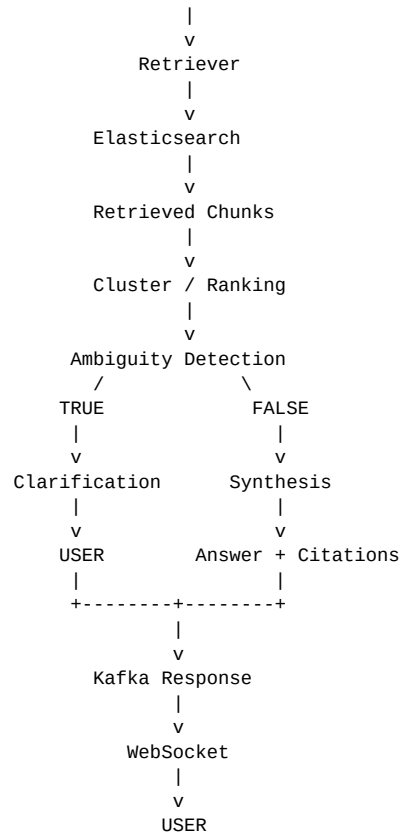
Clarification loop: asks the user for additional information and retries retrieval.

Synthesis: uses retrieved evidence to generate the final answer and citations.

FinalAnswerCollector + Kafka + WebSocket: sends the completed result back to the correct user session.

26. Final Architecture to Keep in Mind





The most important takeaway for your current work is that **retrieval is one layer inside this architecture**. Your present focus should be making the retrieval path reliable first: query → retrieval → relevant chunks → ranking/filtering → evidence → synthesis. The surrounding WebSocket/Kafka/MCP/agent infrastructure can then be layered around that core.